B.Tech. Information Technology

Internet Technology Laboratory – Individual Assignment

Project Title

Design and Development of a Multi-Client Email Service Using C and Network Socket Programming

Duration: 4 Weeks
Mode: Individual Assignment
Programming Language: C
Primary Technology: TCP/IP Socket Programming
Development Assistance: Online LLMs such as ChatGPT are permitted and encouraged
Submission: Source Code + Documentation + Test Evidence + Viva/Demonstration

1. Problem Statement

You are required to individually design and implement a **miniature email service** using the **C programming language and network socket programming**.

The system should consist of a **central email server** and multiple **email clients**. Multiple users should be able to register with the server, log in, and communicate with other registered users by sending and receiving emails through the server.

The project is intended to provide practical understanding of:

- Client-server architecture
- TCP/IP communication
- Socket programming in C
- Concurrent/multiple client handling
- Application-layer communication protocols
- User registration and authentication
- File handling and persistent data storage
- Message formatting and parsing
- Basic networking security considerations
- Debugging and testing of network applications

The objective is **not** to reproduce Gmail or another commercial email system. You are expected to develop a **small, self-contained educational email service** demonstrating the fundamental principles of Internet technology.

2. Learning Objectives

After completing this assignment, a student should be able to:

1. Explain the client-server model.

2. Create TCP server and client programs using C.
3. Establish and terminate socket connections correctly.
4. Design a simple application-layer protocol.
5. Handle multiple clients connected to a server.
6. Implement user registration and authentication.
7. Store and retrieve user and email information.
8. Implement basic email operations.
9. Handle network and application-level errors.
10. Test a distributed application systematically.
11. Use an LLM as a programming assistant while critically evaluating its output.
12. Explain their own source code during a viva.

3. System Requirements

Your system must contain at least the following two components:

A. Email Server

The server should:

- Start and listen on a configurable TCP port.
- Accept connections from multiple clients.
- Register new users.
- Authenticate existing users.
- Maintain a list/database of registered users.
- Receive email messages from clients.
- Store emails on the server.
- Deliver/retrieve emails for registered users.
- Maintain basic user/session information.
- Handle multiple connected clients.
- Provide appropriate responses for invalid requests.
- Continue running even when one client disconnects or encounters an error.

B. Email Client

The client should allow a user to:

- Connect to the server.
- Register a new account.

- Log in using an existing account.
- Compose and send an email.
- Specify one or more recipients.
- Specify a subject.
- Enter a message body.
- View received emails.
- Read a selected email.
- View sent emails.
- Log out/disconnect from the server.

4. Minimum Functional Requirements

The following functionality is **mandatory**.

4.1 User Registration

A new user should be able to register with information such as:

Username

Password

Email Address

The server must check whether the username/email address already exists.

Example:

REGISTER

Username: rahul

Email: rahul@iitlab.local

Password: ********

The server should reject duplicate registrations.

4.2 User Login

A registered user should be able to authenticate with:

LOGIN

Username

Password

The server should respond appropriately to:

- Valid username/password

- Invalid password
- Non-existent username
- Already active session, if your implementation imposes such a restriction

4.3 Sending Email

A logged-in user should be able to send an email.

Example:

From: alice

To: bob

Subject: Internet Technology Lab

Message:

Please complete the socket programming experiment.

The server should verify that the recipient exists.

If the recipient does not exist, an appropriate error message should be returned.

4.4 Receiving Email

A user should be able to retrieve received emails from the server.

At minimum, the client should display:

Email ID

Sender

Subject

Date/Time

Read/Unread Status

The user should be able to select an email and view its complete contents.

4.5 Sent Mail

The client should provide an option to view emails sent by the logged-in user.

Example:

-------- SENT MAIL --------

ID TO SUBJECT

1 bob Meeting

2 amit Lab Assignment

3 neha Project Discussion

4.6 Multiple Clients

The server must support **multiple clients**.

At least **three clients should be demonstrated simultaneously** during the final demonstration.

The implementation may use an appropriate mechanism such as:

- POSIX threads (pthread)
- select()
- poll()
- Another suitable concurrency mechanism

The student must explain why the selected mechanism was used.

5. Suggested Application Protocol

You are required to design a simple **application-layer protocol** between the client and server.

For example, your protocol may contain commands such as:

REGISTER

LOGIN

SEND

INBOX

READ

SENT

LOGOUT

QUIT

A possible interaction could be:

CLIENT → SERVER

LOGIN|alice|password123

SERVER → CLIENT

LOGIN_SUCCESS

Another example:

CLIENT → SERVER

SEND|bob|Assignment|Please complete the assignment.

SERVER → CLIENT

SEND_SUCCESS

Important: These are examples only. Students are expected to design and document their own protocol.

The protocol should clearly specify:

- Request format
- Response format
- Field separators
- Error responses
- Message termination mechanism
- Maximum message size
- Handling of special characters

6. Data Storage

The server must maintain information after a client disconnects.

Students may use simple file-based storage rather than a database.

For example:

users.dat

users.txt

mail/

alice/

bob/

amit/

or another suitable structure.

The minimum persistent information should include:

User information

Username

Password

Email Address

Email information

Email ID

Sender

Recipient

Subject

Message Body

Date/Time

Read/Unread Status

Students may choose their own storage design.

7. Technical Requirements

The project must satisfy the following:

Programming Language

The implementation must primarily be written in **C**.

Networking

Use:

TCP/IP

and standard socket programming APIs such as:

socket()

bind()

listen()

accept()

connect()

send()

recv()

close()

Students should understand and explain every major networking function used.

Server

The server should:

1. Create a socket.
2. Bind it to an IP address and port.
3. Listen for incoming connections.
4. Accept clients.

5. Handle client requests.
6. Send appropriate responses.
7. Manage multiple clients.
8. Store/retrieve information.

Client

The client should:

1. Create a socket.
2. Connect to the server.
3. Authenticate/register.
4. Send commands.
5. Receive responses.
6. Provide a usable menu/interface.
7. Close the connection properly.

8. Error Handling

Your application should gracefully handle situations such as:

- Server unavailable
- Invalid IP address/port
- Connection failure
- Client disconnect
- Invalid username
- Incorrect password
- Duplicate registration
- Sending mail to a non-existent user
- Empty subject/message
- Invalid command
- Malformed request
- File access failure
- Server-side storage failure

The server should **not crash simply because a client sends an invalid request**.

9. AI/LLM Assistance Policy

Students **are permitted and encouraged to use online LLMs such as ChatGPT** during the four-week development period.

The purpose is to teach students how modern software engineers can effectively use AI tools while retaining responsibility for the final implementation.

Students may use an LLM for:

- Understanding socket programming
- Understanding C functions
- Debugging compiler errors
- Designing data structures
- Designing the communication protocol
- Generating code snippets
- Reviewing code
- Finding possible bugs
- Developing test cases
- Improving documentation
- Understanding concurrency mechanisms
- Explaining networking concepts

However, the following principle applies:

Using AI-generated code does not remove the student's responsibility for understanding and explaining the code.

During the viva, the student may be asked to:

- Explain any function in the program.
- Modify a section of the program.
- Identify a bug.
- Explain the client-server communication.
- Explain the protocol.
- Explain how multiple clients are handled.
- Explain where email data is stored.
- Add a small feature.
- Correct an intentionally introduced error.

A student who submits code but cannot explain its operation may not receive full credit.

12. AI Usage Log

Each student must maintain an **AI Usage Log**.

For significant interactions with ChatGPT or another LLM, record:

Date	Task	AI Tool	Prompt/Question	What Was Used	Student Verification
01/10/26	TCP Server	ChatGPT	Asked for explanation of accept()	Explanation	Tested experimentally
04/10/26	Login module	Gemini AI	Debugged login code	Code suggestion	Modified and tested
10/10/26	Multi-client	Claude AI	Asked about pthread	Concept/code	Verified using 3 clients

The purpose is **not to penalize AI usage**. Instead, students are expected to demonstrate responsible AI-assisted software development.

10. Restrictions on AI Usage

Students must **not**:

- Claim that AI-generated code is entirely their own work.
- Submit code that they cannot explain.
- Blindly copy an AI-generated solution without testing it.
- Use AI to fabricate test results.
- Submit fabricated screenshots.
- Ask AI to generate a false project log or false experimental observations.

Students should critically verify AI-generated information because LLM-generated code can contain errors, insecure practices, or incorrect assumptions.

11. Deliverables

Each student must submit the following.

1. Source Code

A complete project containing:

server.c

client.c

*.h

supporting C files

Makefile (recommended)

data files/sample data

Students may organize the project differently if properly documented.

2. Project Report

Recommended structure:

1. Introduction
2. Problem Statement
3. Objectives
4. System Architecture
5. Client-Server Architecture
6. Network Protocol Design
7. Data Storage Design
8. Algorithms/Flowcharts
9. Implementation
10. Multi-client Handling
11. Test Cases
12. Test Results
13. Screenshots
14. AI/LLM Usage
15. Limitations
16. Security Considerations
17. Future Enhancements
18. Conclusion
19. References

3. README File

The README must explain:

- Compilation
- Server execution
- Client execution
- Port number
- Configuration
- Data-file locations
- Example commands
- Known limitations

Example:

gcc server.c -o server -lpthread

gcc client.c -o client

The exact compilation command will depend on the student's implementation.

4. Demonstration

Students must demonstrate the working system to the instructor.

5. Viva

Individual viva will be conducted.